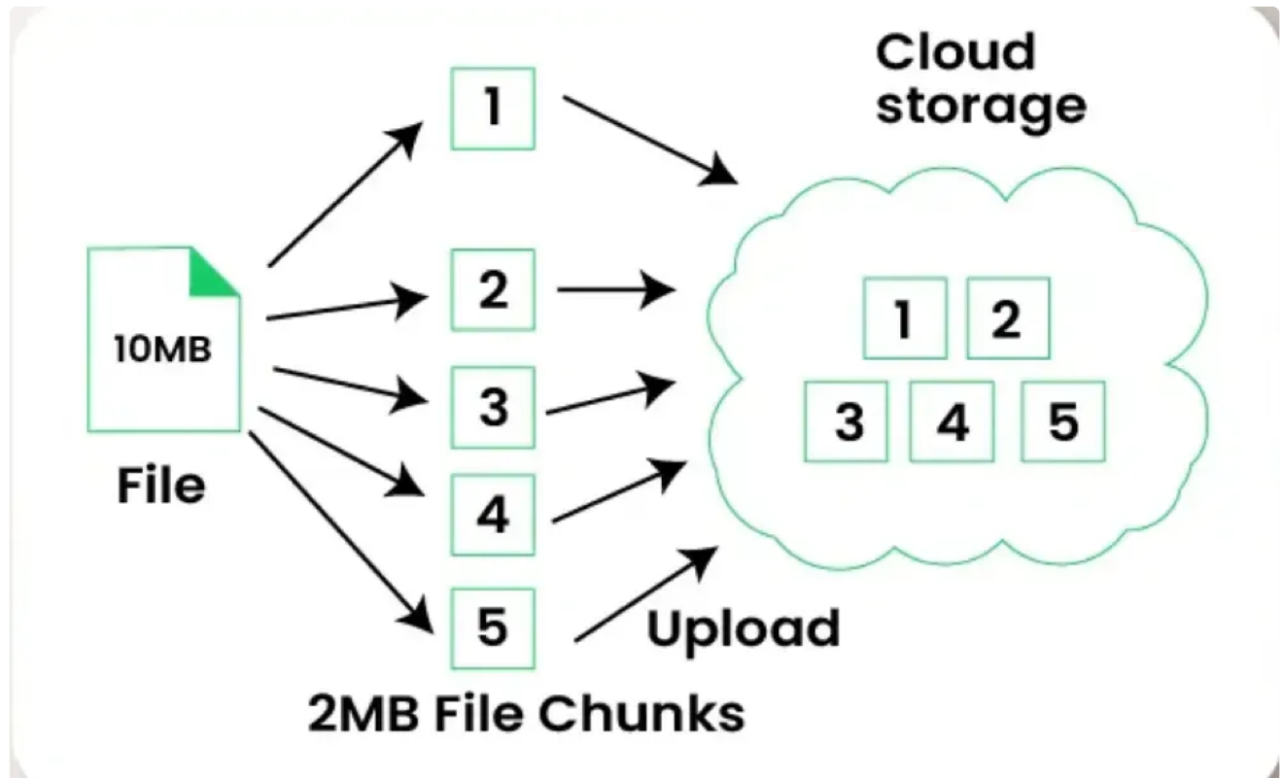


You might have used this file hosting service multiple times to upload and share files or images. System Design Dropbox is a quite common question in the system design round.



1. Requirements Gathering for Dropbox System Design

Functional Requirements:

- The user should be able to upload photos/files.
- The user should be able to create/delete directories on the drive.
- The user should be able to download files
- The user should be able to share the uploaded files.
- The drive should synchronize the data between user all devices.

Non Functional Requirements:

- **Availability:** Availability means what percentage of the time the system is available to take a user's request. We generally mention availability as 5 Nine's, 4 Nine's, etc. 5 Nine's means 99.999% availability, 4 Nine means 99.99% availability, and so on.
- **Durability:** Durability means the data uploaded by the user should be permanently stored in the database even in case of database failure. Our System should ensure the files uploaded by the user should be permanently stored on the drive without any data loss.
- **Reliability:** Reliability means how many times the system gives the expected output for the same input.
- **Scalability:** With the growing number of users our system should be capable enough to handle the increasing traffic.
- **ACID properties:** Atomicity, Consistency, Integrity and Durability. All the file operations should follow these properties.

2. Capacity Estimation for Dropbox System Design

Storage Estimations:

Assumptions:

The total number of users = **500 million**.

Total number of daily active users = 100 million

The average number of files stored by each user = 200

The average size of each file = 100 KB

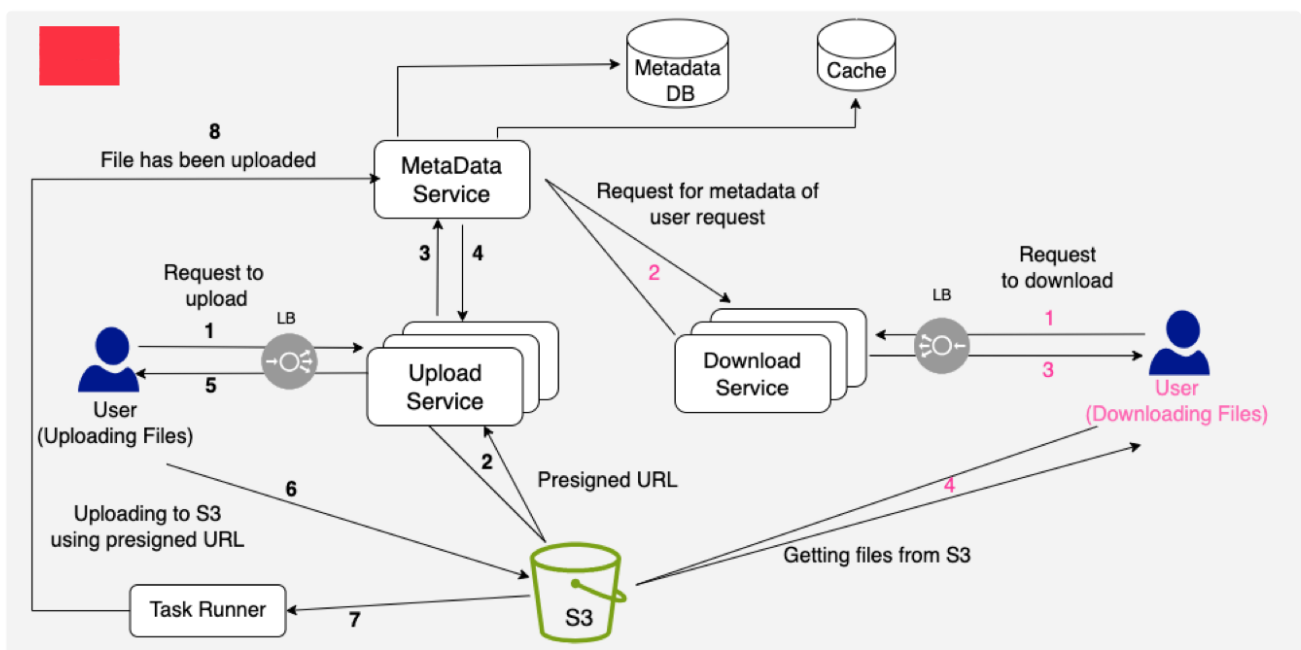
Total number of active connections per minute = **1 million**

Storage Estimations:

Total number of files = 500 million * 200 = **100 billion**

Total storage required = 100 billion * 100 KB = **10 PB**

High - Level Design (HDL) of Dropbox System Design



3.1. User Uploading:

Users interact with the client application or web interface to initiate file uploads. The client application communicates with the Upload Service on the server side. Large files may be broken into smaller chunks for efficient transfer.

3.2. Upload Service:

Receives file upload requests from clients. Generates Presigned URLs for S3 to allow clients to upload directly. Coordinates the upload process, ensuring data integrity and completeness. After successful upload, it updates the Metadata Database with file details. Coordinates the upload process, breaking down large files into manageable chunks if necessary.

3.3. Getting Presigned URL:

The client application requests a Presigned URL from the Upload Service. The server generates the Presigned URL by interacting with the S3 service, creating a unique token for the specific upload operation. These URLs grant temporary, secure access to upload a specific file to a designated S3 bucket. Allows clients to bypass the server for direct communication with the storage layer.

3.4. S3 Bucket:

S3 serves as the scalable and durable storage backend. Presigned URLs allow clients to upload directly to S3, minimizing server involvement in the actual file transfer. The bucket structure may organize files based on user accounts and metadata.

3.5. Metadata Database:

Stores metadata associated with each file, including details like name, size, owner, access permissions, and timestamps. Enables quick retrieval of file details without accessing S3. Ensures that file metadata is consistent with the actual content in S3.

3.6. Uploading to S3 using Presigned URL and Metadata:

The client uses the Presigned URL to upload the file directly to the designated S3 bucket. Metadata associated with the file, such as file name and owner, is included in the upload process. This ensures that the file's metadata is synchronized with its corresponding data in S3.

3.7. Role of Task Runner:

After the file is successfully uploaded to S3, a task runner process is triggered. The task runner communicates with the Metadata Database to update or perform additional tasks related to the uploaded file. This may include updating file status, triggering indexing for search functionality, or sending notifications.

3.8. Downloading Services:

Clients initiate file download requests through the client application. The Download Service queries the Metadata Database for file details. The server's Download Service retrieves metadata from the Metadata Database. Metadata includes information such as file name, size, owner, and access permissions.